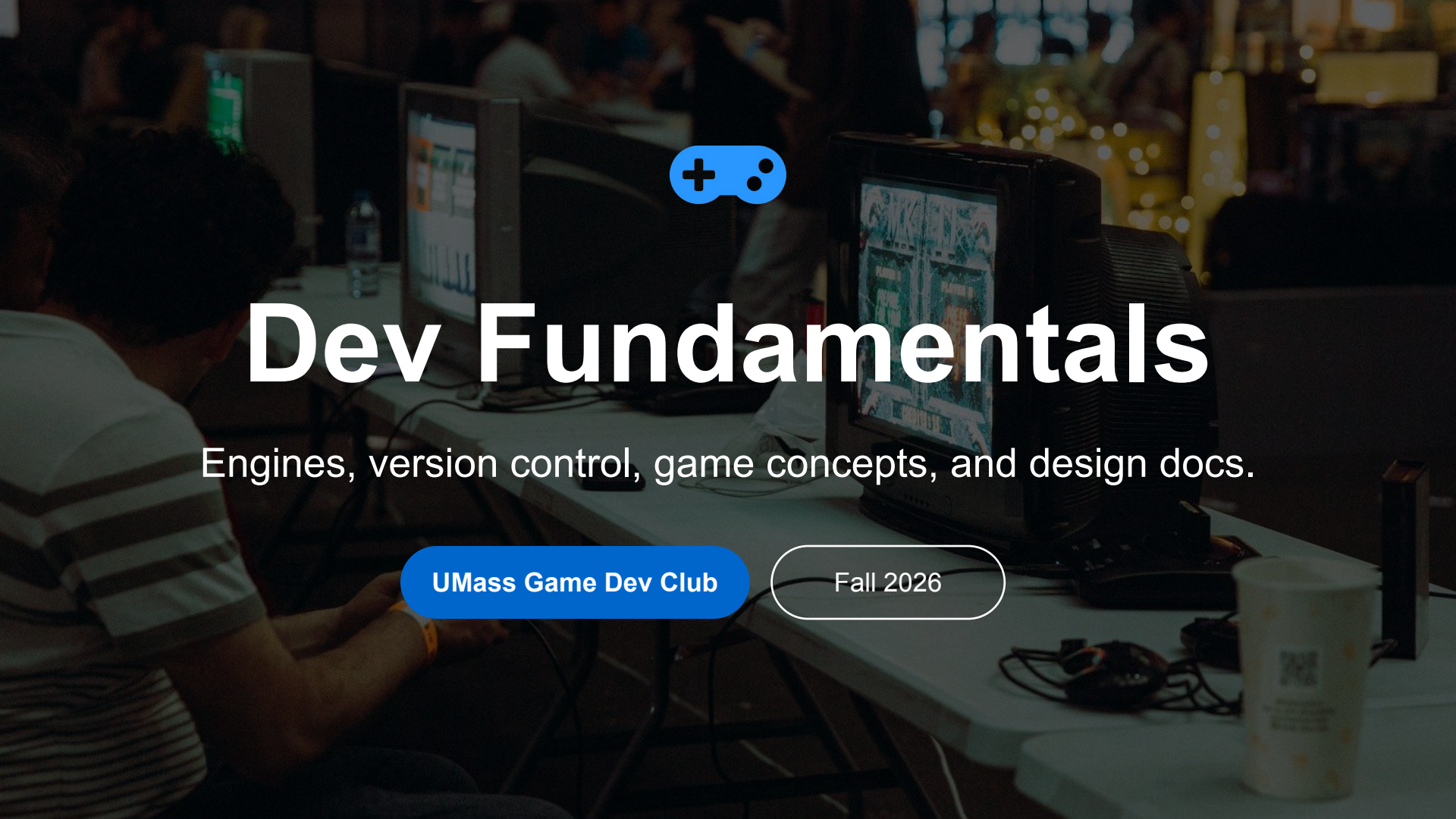


Dev Fundamentals

Engines, version control, game concepts, and design docs.

UMass Game Dev Club

Fall 2026

PRIVATE JAM

LATE OCTOBER

NATIONWIDE COLLAB

TEAMING UP WITH GAME DEV CLUBS ACROSS THE COUNTRY

NEW TEAMS

PROJECT TEAMS START FORMING TONIGHT

EBOARD



Today's roadmap

Five levels. Each one builds on the last.



Level 1

Using an engine

It's just a tool. Learn its quirks.



Level 2

Version control

Never lose work or fight your team.



Level 3

Common questions

The stuff everyone wonders about.



Level 4

Starting a concept

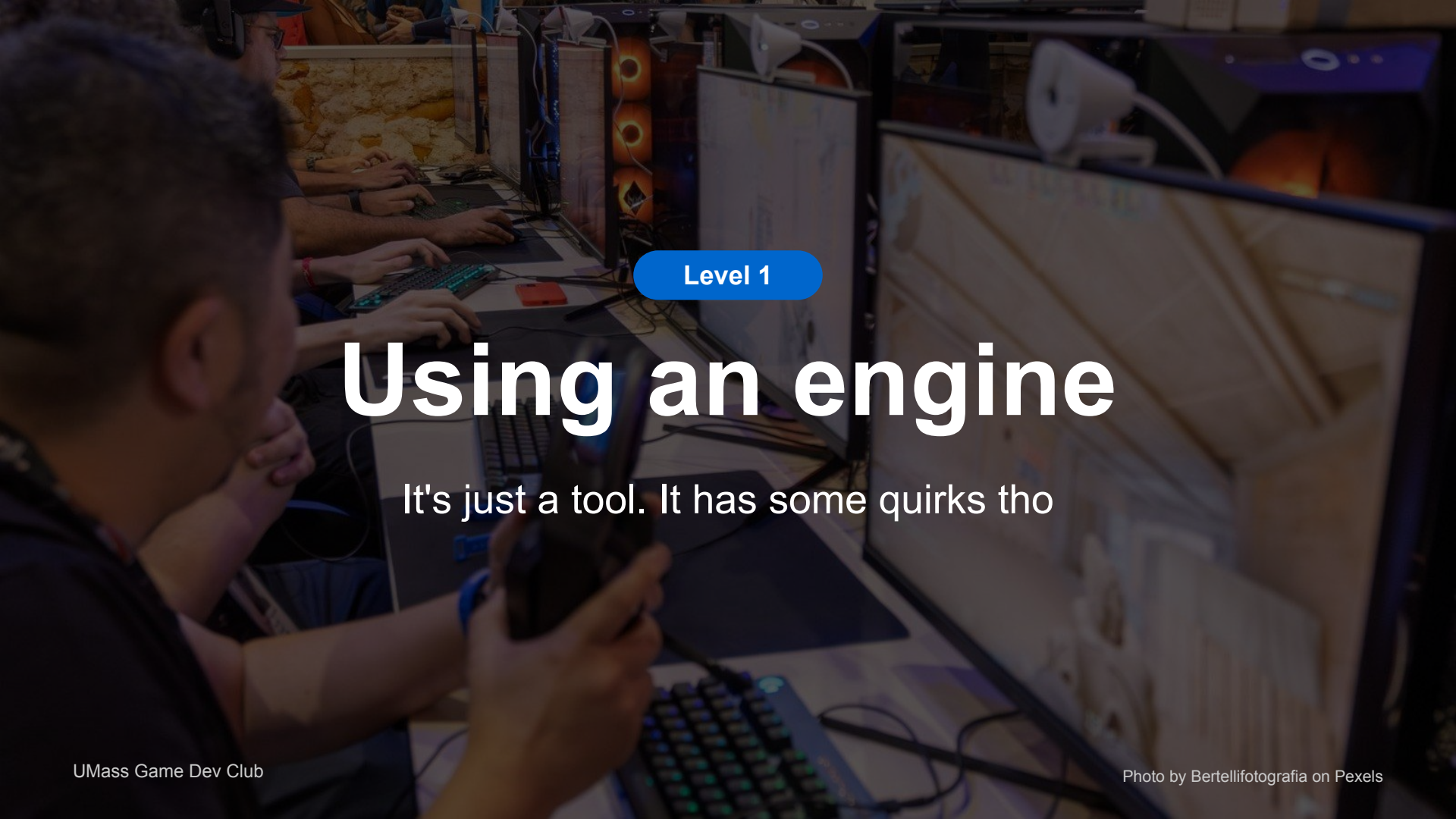
Turn a spark into a plan.



Level 5

Design docs

Get the whole team on one page.



Level 1

Using an engine

It's just a tool. It has some quirks tho

UMass Game Dev Club

Photo by Bertellifotografia on Pexels

What an engine actually does

It's a kitchen. You still cook the meal.

Rendering

Draws every frame on screen

Physics

Collisions, gravity, movement

Input

Keyboard, controller, touch

Audio

Music and sound effects

Editor

Build levels without code

Export

Ship to PC, web, or mobile

Your job: make it fun. The engine's job: everything else.

Engines matter less than you think

It's a tool with its own quirks. Not a religion.

Finishing a game matters. The logo in the editor does not.



Every engine has quirks

You learn them by building. Nobody knows them on day one.



Your skills carry over

Design, scope, iteration, and Git work the same everywhere.



Switching is normal

Plenty of devs use a different engine for each project.

Our club's big three

Three engines, three very different personalities.

Godot

Free and open source

Language
GDScript (like Python) or C#

Best at
Small games, 2D or 3D

Feels like
Light, tidy, node based

Watch out
Fewer tutorials than Unity

GameMaker

The 2D specialist

Language
GML (like C) or Drag and Drop

Best at
Fast 2D prototypes

Feels like
Sprites, rooms, and events

Watch out
No real 3D support

Unity

The all rounder

Language
C#

Best at
3D, mobile, huge asset store

Feels like
Big and modular

Watch out
Heavy editor, lots of options

Where they really differ

None of it is hard and fast. Every one of them can ship a great game.

	Godot	GameMaker	Unity
2D or 3D	Both, 2D is a strength	2D only	Both, 3D is a strength
Editor feel	Tiny, opens in seconds	Light and built for 2D	Heavy, more setup
Cost	Free forever (MIT)	Free to learn, paid to sell	Free under $200K revenue
Source code	Open source	Closed source	Closed source
Consoles	Through third parties	Paid Enterprise plan	Official support
Tutorials	Growing fast	Strong for 2D	The biggest library

Which one should you pick?

Or just use what your team already knows. Any of them works.

I want free, light, and open, in 2D or 3D	→ **Godot**
I want to get a 2D game running fast	→ **GameMaker**
I want 3D, mobile, or the biggest tutorial library	→ **Unity**

Same ideas, different names

Learn the idea once and it works in every engine.

Concept	Godot	GameMaker	Unity
A level or map	Scene	Room	Scene
Anything in the world	Node	Object (instance)	GameObject
Reusable template	Packed Scene	Object	Prefab
Behavior you add	Script on a node	Events (Create, Step, Draw)	Component (script)
Your code	GDScript or C#	GML or Drag and Drop	C#
Asset panel	FileSystem dock	Asset Browser	Project window

Iterate, don't perfect

Every version is playable. No big bang at the end.

1 **Build the smallest playable thing**

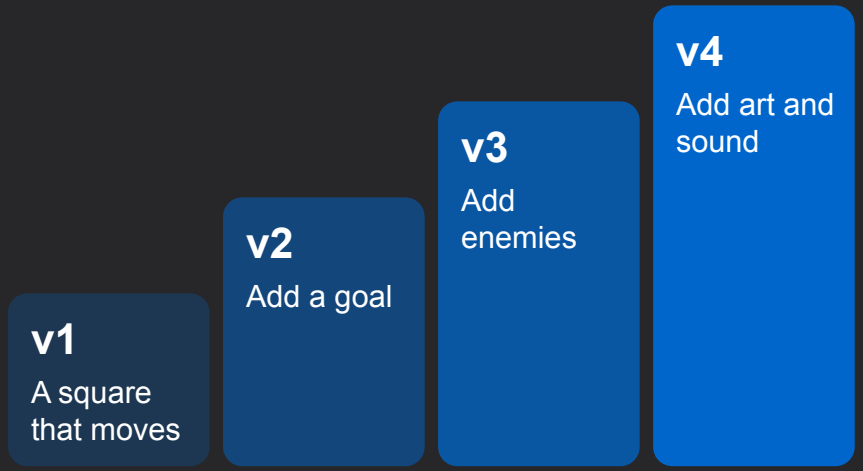
A square that moves counts.

2 **Play it. Change one thing.**

Small changes show what works.

3 **Commit it, then go again.**

Every step is safe to undo.



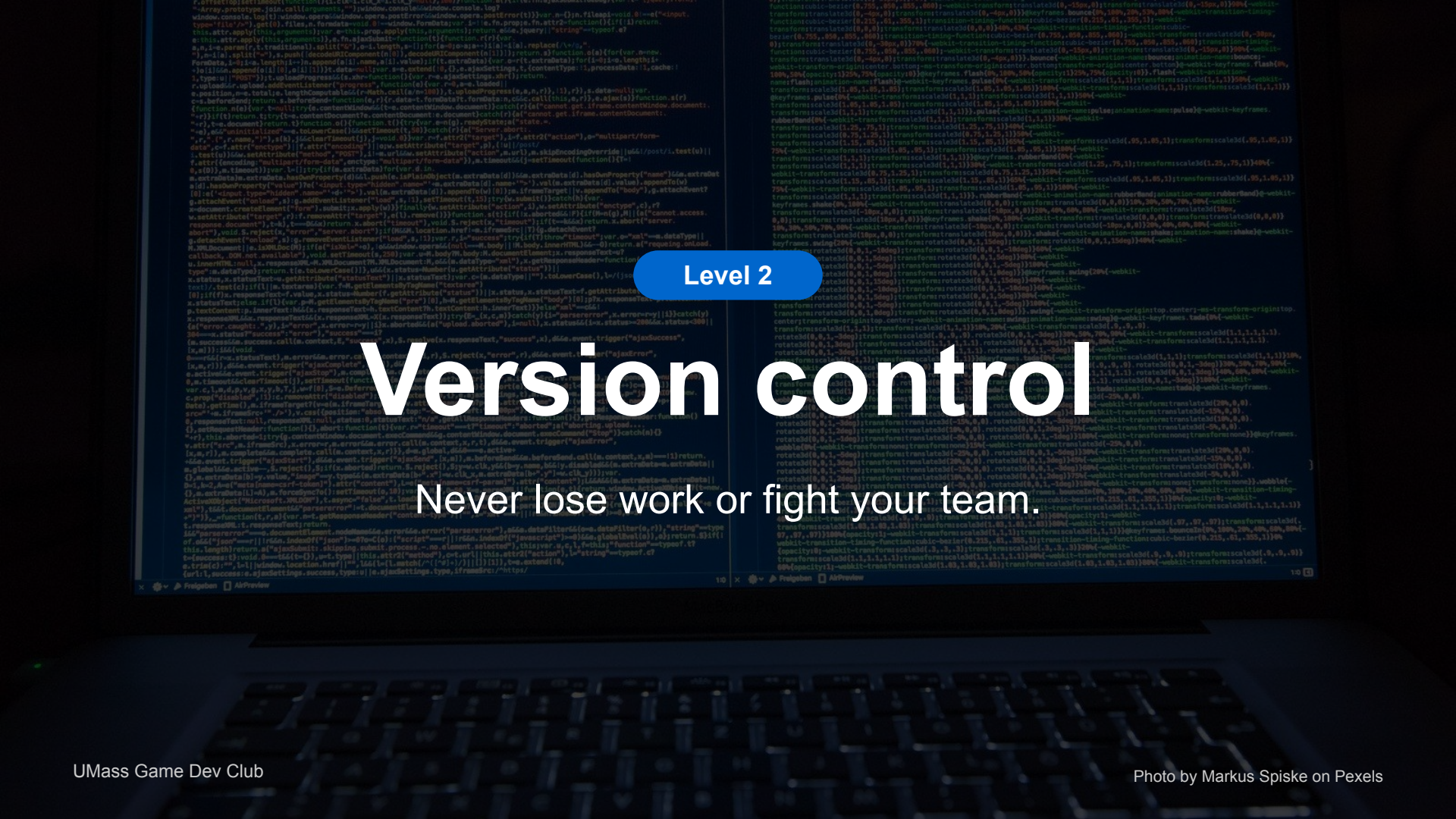


Photo by Markus Spiske on Pexels

Why version control?

It's an undo button for your whole team.



I broke it and can't go back

Rewind to any earlier version in seconds.



Who changed this, and why?

Every change has an author and a note.



We both edited the same thing

Git merges your work instead of overwriting it.

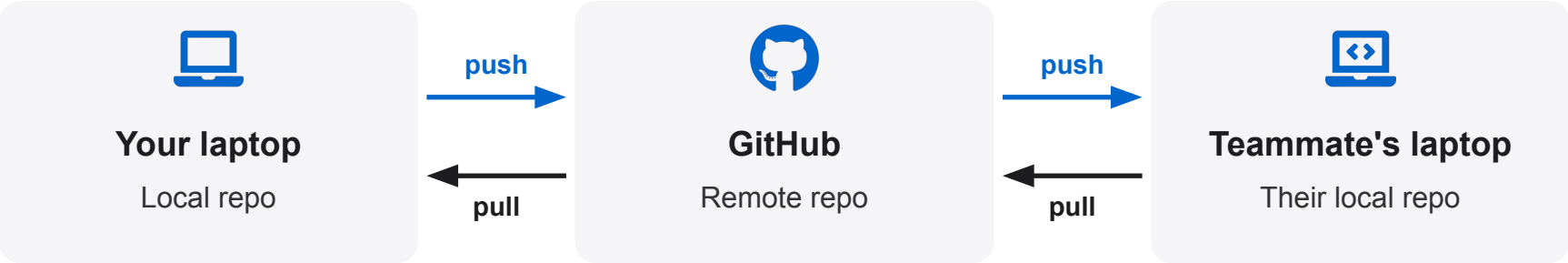


My laptop just died

Your project also lives online, safe and sound.

Git vs GitHub

Git is the tool. GitHub is the place.



Git

The tool on your computer that tracks every change to your project.

GitHub

A website that hosts your repo so the team can share it. Others: GitLab, Bitbucket, Perforce.

The daily loop

Ninety percent of Git is these five steps.

1

Pull

Grab your team's latest changes.

```
git pull
```

2

Work

Open the engine and make your changes.

```
make stuff
```

3

Add

Pick what goes in the snapshot.

```
git add .
```

4

Commit

Save a snapshot with a clear message.

```
git commit
```

5

Push

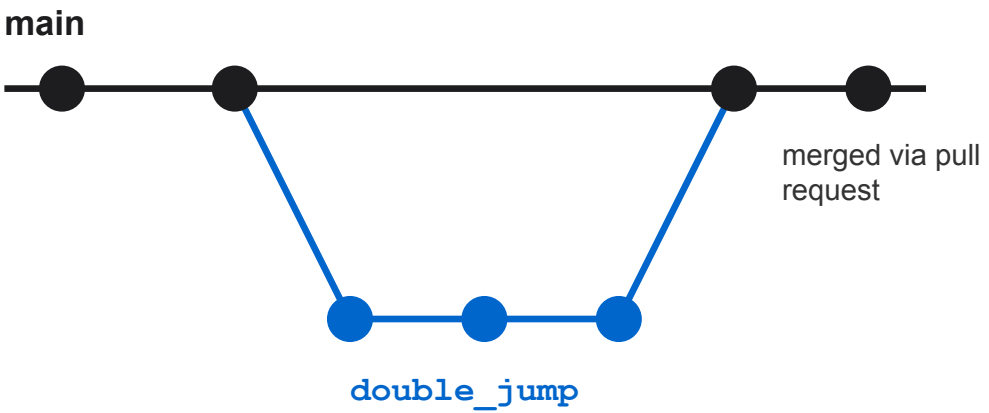
Share it with the team.

```
git push
```

Commit small. Commit often. Write 'Add double jump', not 'stuff'.

Branches keep main safe

Experiment on a branch. Merge it when it works.



Keep main clean

It should always run. Never test half finished work there.

One branch per feature

Jump, menu, boss fight. Each gets its own branch.

Merge with a review

Open a pull request so a teammate checks it first.

Game repos are special

Your engine makes a lot of junk. Keep it out of Git.

Ignore it

The engine regenerates these, so they only bloat your repo.

Unity: Library, Temp, Obj, Builds

Godot: the .godot folder

GameMaker: .resource_order files

Track it

Anything you'd need to rebuild the project on a new laptop.

Scenes, rooms, scripts, settings

Unity: keep every .meta file

Unity: set Force Text serialization

The .gitignore file itself

Use LFS

Big binary files fill your repo fast and never shrink.

Textures, audio, models, video

Set it up before big commits

Track by file type, not folder

Start from GitHub's ready made .gitignore template for your engine, then tweak it.

When Git says conflict

It's normal. Here's how to handle it.

```
<<<<<<< HEAD
speed = 5
=======
speed = 8
>>>>>>> double_jump
```

Git shows both versions. You choose what stays.

Game tip: scenes and prefabs conflict the most. One person per scene at a time, and split levels into prefabs.

1. **Don't panic**
   Conflicts are normal on teams.
2. **Pull often**
   Small, frequent syncs mean small conflicts.
3. **Talk first**
   Ask whoever changed it what they meant.
4. **Fix and commit**
   Keep the right lines, delete the markers.



Level 3

Common questions

The stuff everyone wonders about.

Photo by William Fortunato on Pexels

Questions everyone asks

Every game dev started as a beginner.

Do I need to know how to code?

Not on day one. GDScript and drag and drop are gentle. Logic matters more than syntax.

Is there a place for artists and musicians?

Yes! Games need art, audio, writing, UI, and playtesting as much as code.

How long does a game take?

A jam game: 48 hours. A small real game: a semester. A dream game: years.

Where do I get art and sound?

Make your own or grab free packs from itch.io, Kenney, or OpenGameArt. Check licenses.

How do I find a team?

Come to coworking, hang in the Discord, join a jam. Teams form around people who show up.

Do I need a powerful computer?

Not for Godot or GameMaker. Unity wants a bit more, especially for 3D.

Stuck? Do this first

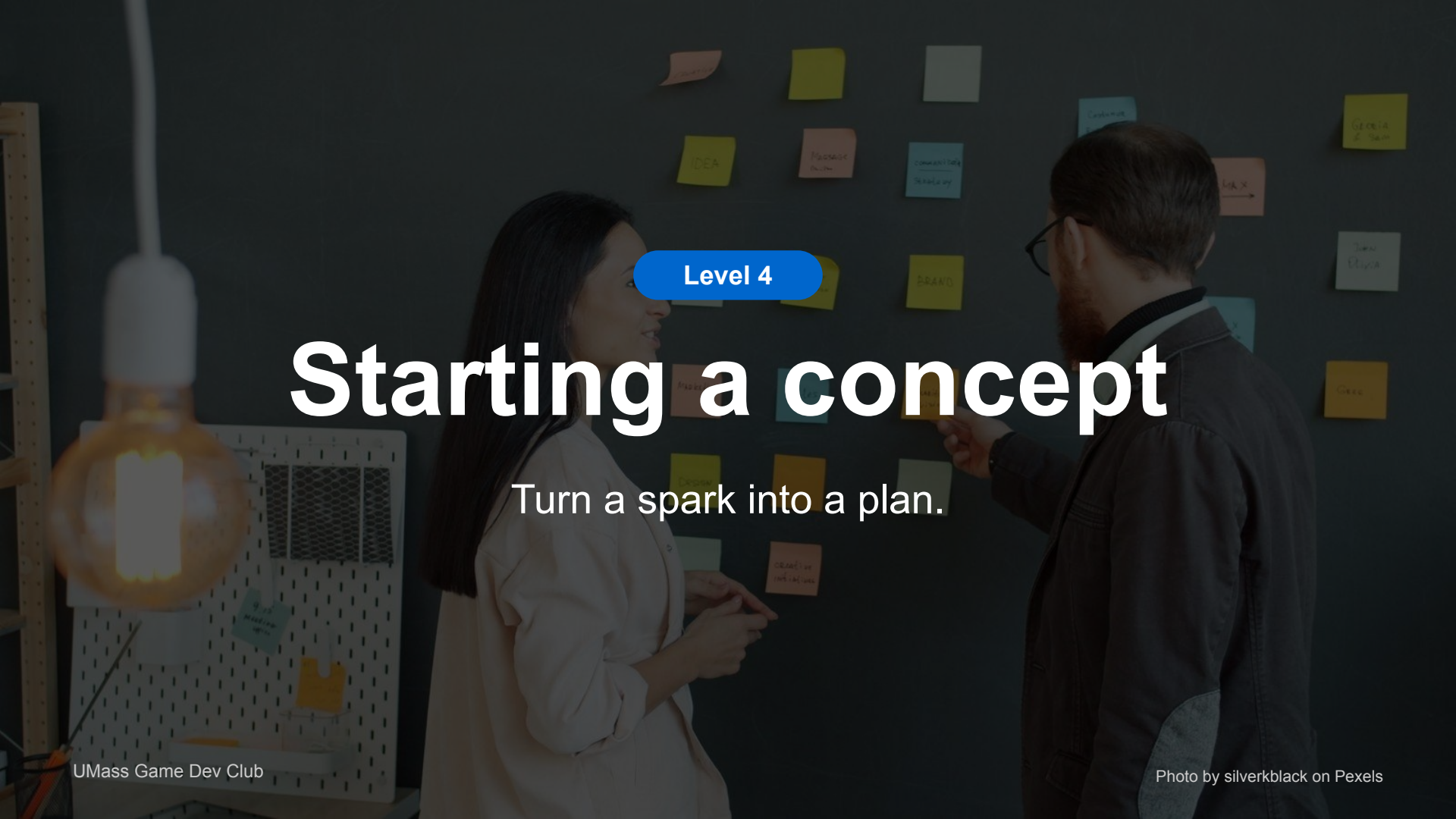
Most bugs fall to the first four steps.

1. **Read the error.** All of it, top to bottom.
2. **Search the exact message.** Add your engine's name.
3. **Print your values.** See what is really happening.
4. **Shrink it.** Remove things until the bug vanishes.
5. **Ask for help.** Show what you tried.

A good help request

- What I expected
- What actually happened
- What I already tried
- A screenshot or the code

People help faster when you make it easy.



Level 4

Starting a concept

Turn a spark into a plan.

UMass Game Dev Club

Photo by silverkblack on Pexels

Start with the verb

Ask what the player does, not what the story is.

Fuzzy idea		Playable pitch
A game about time travel	→	**Rewind 5 seconds to undo your mistakes**
A cozy farming game	→	**Grow and sell crops before the storm hits**
A horror game	→	**Sneak past a monster that hunts by sound**

Verbs become mechanics. Mechanics become games.

The one sentence pitch

Fill in the blanks. Then test it.

You play as a ____ who must ____ before ____.

Example

You play as a courier who must deliver packages across a flooding city before the water rises.

- Can a stranger picture it?
- Is the goal clear?
- Is there tension?

Find your core loop

Every fun game has a loop you repeat. Example: a platformer.



If the loop isn't fun with plain cubes and no art, polish won't save it.

Scope is the real boss

Scope kills more games than bad ideas.

Game jam

48 hours. 1 mechanic.

Semester project

1 mechanic done well. 3 to 5 levels.

Dream game

Years and a real team. Earn it later.

Scope rules

1. Cut your feature list in half. Then cut it in half again.
2. Pick 3 pillars. Say no to everything else.
3. Finish a small game before starting a big one.

Prototype before you polish

Find the fun before you make it pretty.

1

Sketch

Paper or whiteboard.
Ten minutes.

→

2

Greybox

Cubes and squares.
No art.

→

3

Playtest

Watch 3 people play.
Stay quiet.

→

4

Decide

Keep it, change it, or kill it.

Find the fun first

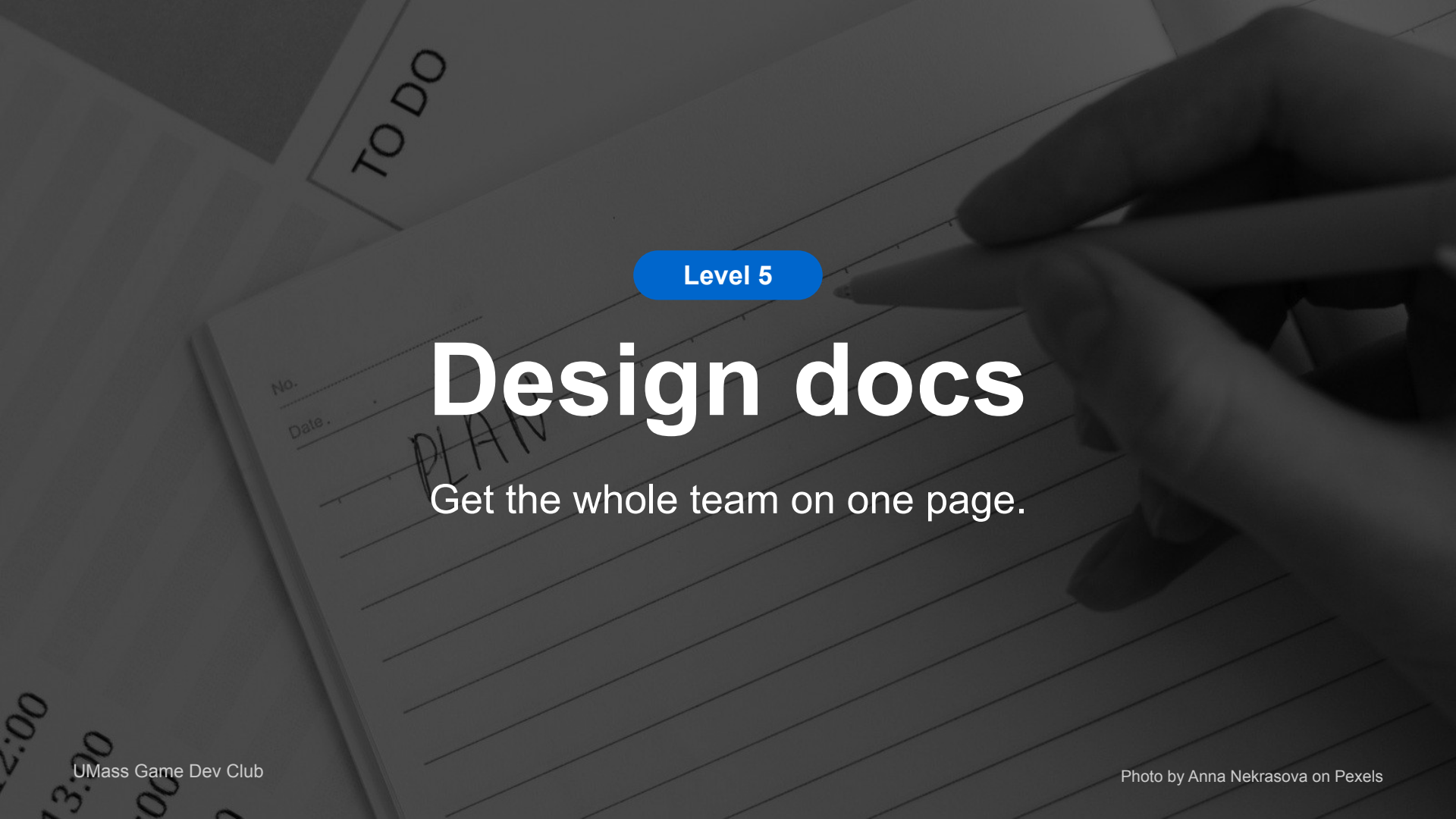
Make it playable before you make it pretty.

Watch, don't explain

Silence shows you where players get lost.

Kill ideas early

It's a win. You just saved yourself weeks.



Level 5

Design docs

Get the whole team on one page.

Photo by Anna Nekrasova on Pexels

What's a design doc?

Your team's shared brain. It changes as the game does.



Everyone builds the same game

No more five people imagining five different games.



New members catch up fast

Read one page instead of asking ten questions.



Arguments get settled

Point at the doc, not at each other.

Start with a one pager

If it doesn't fit on one page, you're not ready for a longer one.

1. **Title and pitch**
   One sentence, no jargon
2. **Genre and platform**
   Who plays it, and on what
3. **Core loop**
   The 30 second cycle
4. **Pillars**
   Three words that guide choices
5. **Controls**
   What each button does
6. **References**
   Games that inspire you
7. **Scope**
   What's in and what's out
8. **Team**
   Who owns what

Growing it into a full GDD

Add a section only when the team needs it.

Overview
Pitch, audience, pillars

Gameplay
Mechanics, controls, rules

Content list
Levels, enemies, items

Art direction
Style, mood, references

Audio direction
Music and sound feel

UI and UX
Menus, HUD, flow

Tech notes
Engine, tools, limits

Schedule
Milestones and owners

Risks
What could sink the project

Example: a real one pager

A made up game with a real level of detail.

High Water

Pitch	You play as a courier delivering packages across a flooding city before the water rises.
Genre	2D platformer for PC. 10 minute sessions.
Core loop	Run, jump, deliver, buy gear, repeat.
Pillars	Fast. Tense. Readable.
Scope	In: 5 levels, 3 upgrades. Out: multiplayer, cutscenes.
Team	2 programmers, 1 designer, 1 artist, 1 audio.

What to notice

Short sentences

Nobody reads paragraphs.

An out list

Saying no is a feature.

One page total

That's the whole point.

Doc habits that work

The best doc is the one people actually open.

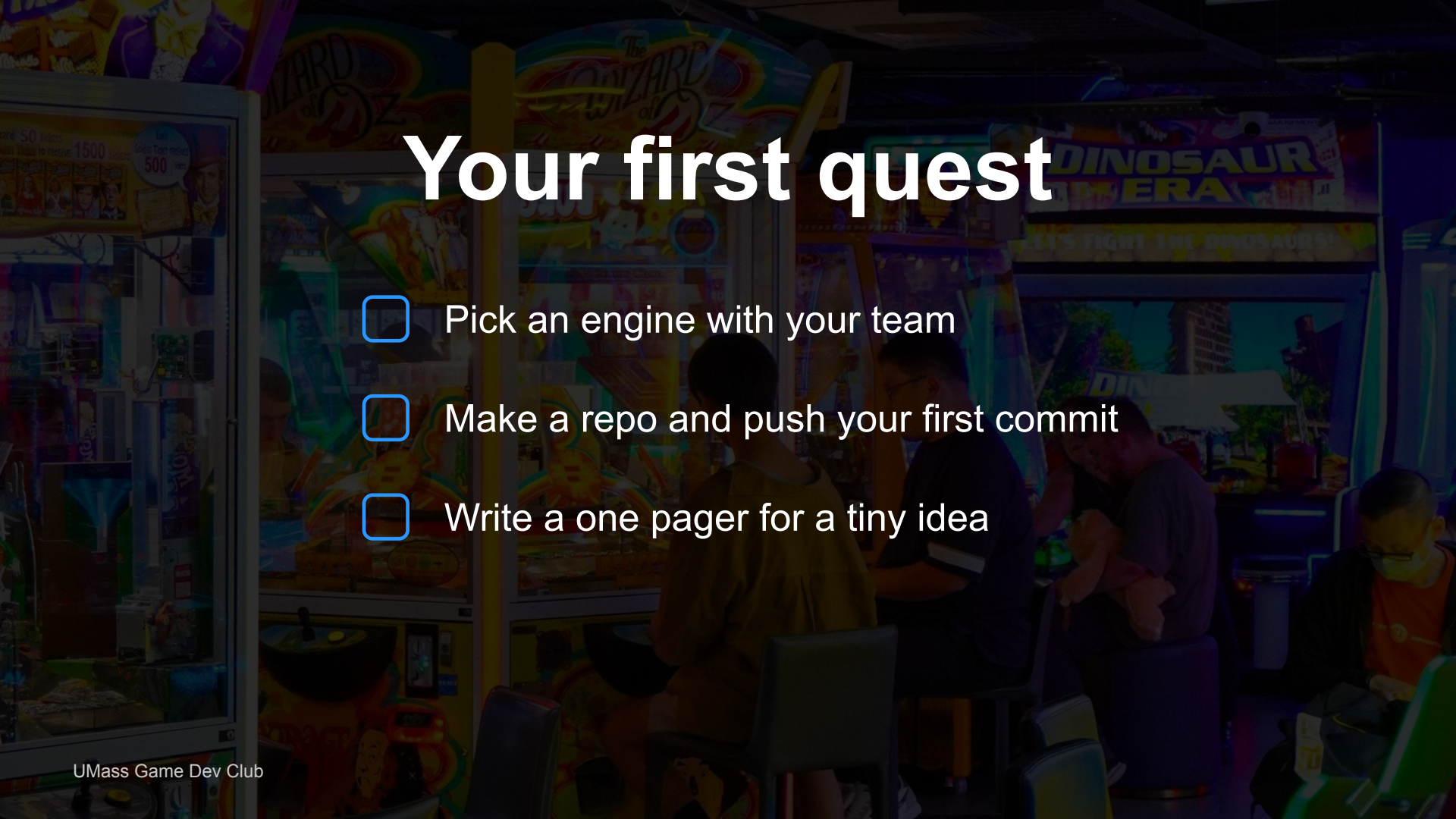
✓ Do this

- Keep it short and visual
- Give it one owner
- Date your changes
- Store it where everyone can find it

✗ Skip this

- A 60 page novel nobody reads
- Letting it go stale
- Hiding copies in five places
- Treating it as law after playtests say no

Where to keep it: a shared Google Doc, Notion, or a markdown file in your repo.


Your first quest
Pick an engine with your team
Make a repo and push your first commit
Write a one pager for a tiny idea
UMass Game Dev Club

WHAT DO YOU WANT TO LEARN ABOUT???



INSTRUCTIONS

1. Go to the Elements tab on the left
2. Click on the Sticky Notes tab
3. Pick a shape and add some text!

PS: you can doodle too if you want to! :)

JOIN THE JAM!!!